# BioVision Intelligence

Project explanation — architecture, warehousing reasoning, data provenance and model metrics.
Generated from the live backend on 2026-10-28.

A walk-through of what the project is, the reasoning behind every major design decision, the data warehousing concepts we picked and why, the exact rows / columns / target variables we train on, and how the machine-learning and LLM layers plug into the warehouse.

Numbers in this document are live — pulled from the running database and the `/overview`, `/cv/report` and `/analytics/*` endpoints on `2026-10-28`.

## 1. What BioVision is (and isn't)

BioVision is a **multi-omics research workspace**. It unifies four things that researchers usually have to open in four different windows:

1. A **dimensional data warehouse** of genes, mutations, proteins, RNA expression, samples and images.
2. A **live multidimensional OLAP engine** with the five canonical operations (roll-up / drill-down / slice / dice / pivot).
3. **Classical and deep machine-learning models** that read from the warehouse: a CNN with Grad-CAM, Random Forest / Decision Tree / Logistic Regression / SVM / KNN classifiers, K-Means and DBSCAN clustering, Apriori / FP-Growth association mining, and an Isolation-Forest anomaly scorer.
4. A **DNA editing bench** and a **natural-language investigation layer (Nexus)** that lets a researcher ask a question in English, get a planned sequence of allowlisted analytical tools, and receive a grounded explanation of the result.

It is **not** a clinical system, not a biobank, and not a production diagnostic tool. Every page carries that disclaimer at the foot.

## 2. Logical reasoning — why each piece exists

### 2.1 Why a warehouse at all?

Biomedical datasets live in different shapes. Ensembl ships gene coordinates as GFF, UniProt ships proteins as flat TSVs, ClinVar ships variants as VCFs, GTEx ships RNA counts as compressed matrices, MedMNIST ships images as `.npz`.

Giving a researcher a tool that merges all of these at query time — on every page load — is slow, fragile, and prevents cross-cutting analysis. The classical answer, from Kimball & Ross (*The Data Warehouse Toolkit*), is to pre-ingest everything into a single **dimensional model** with:

- **Conformed dimensions** — one `DIM_GENE` row per gene, shared by every subject area (RNA, protein, mutation). A `TP53` row is the same `TP53` row for the mutations view as for the expression view.
- **Surrogate keys** — numeric `gene_key` not the string `"ENSG00000141510"`, so joins are cheap and type-safe.

- **A single fact table** — one row per *observation* of a biological

entity, in a given sample, under a given condition, at a given time. All measures live on that row.

- **Snowflake extensions** only where the hierarchy is real — a gene

belongs to a chromosome, a chromosome to a genome, a genome to an organism; we materialise that chain so a *roll-up* actually corresponds to a meaningful biological grouping.

The dimensional model is the thing that lets us treat *"compare TP53 expression across tissues for healthy vs cancer samples"* and *"show the mutation-type distribution in the current dataset"* as two instances of the same operation: a `SELECT … GROUP BY` over the fact table joined to a dimension.

## 2.2 Why OLAP (and why in 3-D)?

Classical OLAP vocabulary (Codd, Codd & Salley 1993; Chaudhuri & Dayal 1997) is the shortest known description of what a researcher wants:

| Operation | Analytical intent | How we implement it |
|---|---|---|
| Roll-up | Aggregate along a hierarchy to a coarser level | Replace one `GROUP BY` column with its parent column |
| Drill-down | Expose a finer level of detail | Replace a `GROUP BY` column with its child column |
| Slice | Fix one dimension to a single value | Add a `WHERE dim = v`; draw a translucent plane in cube |
| Dice | Filter sub-cube across several dimensions | Add several `WHERE … IN (…)` clauses |
| Pivot | Change the analytical perspective (rotate dimensions) | Swap the `(x, y, z)` axis mapping; cube quarter-turns |

The **3-D cube renderer** is not a decoration. It makes the dimension-swap explicit: the user watches the axes relabel and the voxels re-tint when they change the metric. The alternative — tucking the query in a hidden form — hides the thing we want the researcher to understand.

## 2.3 Why keep the LLM at the edge?

The Nexus investigation layer uses Groq as its planner + explainer, not as its executor. The LLM:

- **Can** choose one of 7 allowlisted analytical tools.
- **Can** read structured tool outputs and summarise them in prose.
- **Cannot** issue arbitrary SQL, run Python, or access the model

artefacts.

Everything the user sees as a *number* comes from a backend aggregation; everything the user sees as *prose* is clearly labelled as model-generated interpretation. If the model goes offline, the system falls back to a rule-based planner that uses the same tool allowlist, and a rule-based explainer that walks the structured output. The research disclaimer stays in place in both paths.

## 2.4 Why the Newsprint → Intelly design pivot?

Early versions of the UI used an editorial language (big serif, hard borders, uppercase labels). That reads well in a conference paper but poorly as a *working* dashboard: too much vertical whitespace, too little information above the fold. The reference we match now is the "intelly" healthcare SaaS mock: **compact black floating sidebar, warm cream canvas, pastel category cards, large soft radii, Inter for UI typography, Fraunces as a selective display face.** It is denser, friendlier, and uses colour semantically (pink = genomics, yellow =

transcriptomics, green = proteomics, blue = vision/ML, purple = primary action).

# 3. Architecture — a one-page map




# 4. Data warehousing concepts, applied

## 4.1 The star schema

The hierarchy highlighted on the left is the **snowflake** — a deliberate exception to the strict star. It only exists because a gene really does belong to a chromosome, and a chromosome to a genome. We did **not** snowflake elsewhere because every other dimension is a flat conformed lookup.

## 4.2 Dimension table inventory (live counts)

| Dimension | Rows | Columns | Grain |
|---|---|---|---|
| `dim_organism` | 1 | 4 | one species (Homo sapiens, taxonomy 9606) |
| `dim_genome` | 1 | 5 | one assembly (GRCh38) |
| `dim_chromosome` | 24 | 5 | one chromosome per genome (chr1 … chr22, X, Y) |
| `dim_gene` | 20 | 9 | one gene (gene_name, biotype, start/end position, strand) |
| `dim_sample` | 569 | 6 | one patient / donor sample (WDBC cohort + reference) |
| `dim_rna` | 280 | 10 | one RNA observation (gene × tissue × condition) |
| `dim_protein` | 20 | 10 | one protein (length, MW, abundance, EC, localisation) |
| `dim_mutation` | 33 | 10 | one catalogued variant (ClinVar / COSMIC curation) |
| `dim_disease` | 7 | 4 | one disease category |
| `dim_image` | 120 | 8 | one biomedical image patch |
| `dim_time` | 96 | 7 | one day on the ingestion calendar |

Row counts come from `SELECT COUNT(*)` on the live DB (`biovision.db`).

## 4.3 Fact table

`fact_biological_observation` — **8 535 rows**, **16 columns**. Grain: one observation of one biological entity, for one sample, under one condition, on one date.

Dimension foreign keys (surrogate):

```
sample_key   gene_key   genome_key   rna_key   protein_key
mutation_key disease_key image_key   time_key
```

Measures:

```
expression_value    (NUMERIC)     TPM-style quantity
protein_abundance   (NUMERIC)     normalised abundance signal
mutation_count      (INTEGER)     count of variants observed
cv_score            (NUMERIC)     computer-vision score 0..1
ml_prediction       (STRING)      model prediction label
ml_confidence       (NUMERIC)     model confidence 0..1
```

The additivity rules for measures are respected in the OLAP engine: `expression_value` and `cv_score` are averaged, never summed; `mutation_count` is summed; `ml_confidence` is averaged only where the aggregate makes sense.

### 4.4 Categorical breakdown of the warehouse

Pulled live from the DB:

**Mutation types** — SNP 26 · DEL 3 · SUB 2 · INS 2 (total 33)

**Mutation consequences** — missense 18 · nonsense 8 · frameshift 4 · amplification 1 · fusion 1 · in-frame_del 1

**RNA observations by condition** — healthy 140 · diseased 140

**RNA tissues covered** — Breast · Lung · Colon · Skin · Brain · Ovary · Blood

**Sample conditions** — healthy 357 · diseased 212

**Disease catalogue (7)** — Invasive Breast Carcinoma · Lung Adenocarcinoma · Colon Adenocarcinoma · Cutaneous Melanoma · Glioblastoma Multiforme · Ovarian Serous Cystadenocarcinoma · Healthy Control

## 5. Data sources — what was ingested from where

| Source | Modality | Dimension | Status |
|---|---|---|---|
| Ensembl | Gene coordinates | `DIM_GENE`, `DIM_CHROMOSOME`, `DIM_GENOME` | Local ingest (GRCh38 subset) |
| UniProt | Protein sequence + metadata | `DIM_PROTEIN` | Local ingest |
| ClinVar | Curated clinical variants | `DIM_MUTATION` | Local ingest |
| COSMIC | Somatic cancer mutations | `DIM_MUTATION` | Local ingest |
| GTEx | RNA expression (healthy) | `DIM_RNA` | Local ingest |
| TCGA-BRCA | RNA expression (cancer) | `DIM_RNA`, `DIM_SAMPLE` | Local ingest |
| UCI Wisconsin WDBC | 30 cell-nuclei features / 569 patients | `DIM_SAMPLE` + sample_features | Local ingest |
| MedMNIST / BreastMNIST | 28×28 ultrasound patches | `DIM_IMAGE` | Local ingest |

BioVision is **not currently connected** to a live external pipeline. Everything above the warehouse is the state of the last local ETL run.

## 6. Machine-learning models — target variables, features, splits, metrics

Every number in this section comes from `/cv/report` and `/analytics/*` on the running instance.

### 6.1 Vision — BreastMNIST CNN with Grad-CAM

- **Dataset**: MedMNIST / BreastMNIST — real breast ultrasound patches

derived from Al-Dhabyani et al. 2020.

- **Classes (target)**: `malignant` vs `normal_benign` (binary).

- **Input shape**: 1 × 28 × 28 (single-channel grayscale).
- **Model**: 2-layer convolutional net —

`Conv2d(1→16, 3×3) → MaxPool → Conv2d(16→32, 3×3) → MaxPool → Linear → 2 logits`.

- **Training split**: `n_train = 546`, `n_val = 78`, `n_test = 156`.
- **Hyper-parameters**: `epochs = 8`, `batch_size = 64`, `optimizer = Adam`, lr = 1e-3`.
- **Live test metrics**:
- `test_accuracy = 0.7692`
- Confusion matrix (rows = true, columns = predicted):

`[[9, 33], [3, 111]]` (9 malignant correct, 33 malignant missed, 3 false positives, 111 benign correct)

- **Explanation**: Grad-CAM is computed from the gradient of the predicted class with respect to the last convolutional feature map; the heatmap is overlaid on the input image. The frontend shows the real inference result side-by-side with its Grad-CAM attention.

## 6.2 Classification — WDBC with 5 classifiers

- **Dataset**: Wisconsin Diagnostic Breast Cancer (UCI).
- **Rows / features**: `569 samples × 30 features` — the 30 real cell-nuclei features (mean / standard error / worst of radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension).
- **Target variable**: `biological_condition` (binary — `healthy` vs `diseased`), labelled from the WDBC `target` column.
- **Train / test split**: 75 / 25 stratified, `random_state = 42` (so 426 train / 143 test).
- **Models compared**:

| | Model | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|---|
| | Random Forest (250) | 0.9580 | 1.0000 | 0.8868 | 0.9400 |
| | Decision Tree | 0.9580 | 0.9796 | 0.9057 | 0.9412 |
| | *best* on this split | DecisionTree | | | |

Random Forest, Logistic Regression, SVM and KNN are additionally trained; the `/analytics/classification` endpoint returns all five and the UI picks whichever the user clicks.

- **Feature importances**: returned per-feature from the RF model, surfaced in the Data Mining Studio.
- **Live inference**: the Try-it form on the Classification tab issues a real `POST /analytics/classification/predict` call with the user's input vector.

## 6.3 Clustering — K-Means + DBSCAN over WDBC features

- **Input**: same 30-feature matrix as classification, standardised via `StandardScaler`.
- **K-Means**: `k = 3`, `n_init = 10`, `random_state = 42`.
- **Live metrics**:
- silhouette = `0.3144`
- inertia = `10 061.80`

- cluster sizes: {0: 110, 1: 359, 2: 100}
- **DBSCAN**: `eps = 1.2`, `min_samples = 3`. On this dataset DBSCAN

flags all 569 points as noise (`n_clusters = 0`), which the UI reports honestly rather than hiding.

- **Visualisation**: 2-D PCA scatter; principal components explain

`44.3 %` and `19.0 %` of the variance respectively.

## 6.4 Anomaly detection — Isolation Forest

- **Model**: `IsolationForest(contamination = 0.15, random_state = 42)`.
- **Input**: same WDBC feature matrix.
- **Live result**: `n_samples = 569`, `n_anomalies = 86`

(matches the contamination parameter).

- **Top flagged samples** are returned sorted by score; the Studio

renders them in accent red and cross-highlights the matching table row.

## 6.5 Association rule mining — Apriori / FP-Growth

- **Transactions**: 569 — one transaction per sample, items are the

dataset-derived tokens (e.g. `mut:BRAF:SNP`, `expr:TP53:high`, `disease:breast_cancer`).

- **Parameters**: `min_support = 0.35`, `min_confidence = 0.70`,

`max_itemset_length = 4`.

- **Output**: frequent itemsets with support, confidence and lift.

Example on the current DB: the itemset `{BRAF SNP, BRCA1 DEL, CDKN2A SNP, KRAS SNP}` reaches `support = 1.0` on this cohort.

- **Interpretation guard**: the Studio prints *"these are

dataset-derived associations, not causal biological claims"* above the rule table.

## 6.6 Variant-effect scorer (heuristic, not ML)

Transparent heuristic combining:

- a coarse consequence weight (silent / missense / nonsense / etc.),
- a conservation-window prior,
- a disease-context prior from `DIM_DISEASE`,
- an exact-match bonus against curated ClinVar / COSMIC hotspots.

The scorer returns a score, a band, and a free-text `rationale` listing every contribution. It is labelled *heuristic* in the UI; it is **not** a replacement for AlphaGenome.

## 6.7 LLM — Groq, grounded, allowlisted

The Nexus layer uses `llama-3.3-70b-versatile` on Groq when `GROQ_API_KEY` is exported on the backend. It does two things:

- **Plan**: given a question + the tool allowlist (7 tools), emit a

JSON plan. Params are typechecked and gene names are validated against `dim_gene` before execution.

- **Explain**: given the structured step outputs, write a prose

explanation ≤ 120 words that *cannot* introduce numbers absent from the structured results.

Without a key, both steps fall back to deterministic rule-based behaviour. The provider status is printed honestly on the Nexus page.

# 7. The five OLAP operations in SQL

The frontend holds a 10-field state object; each operation mutates one field and triggers one call to `GET /olap/cube`. Example per operation:

```
-- Roll-up: gene → chromosome
SELECT c.chromosome_number AS y,
       t.year              AS x,
       d.disease_name      AS z,
       COUNT(*)            AS metric
FROM   fact_biological_observation f
JOIN   dim_gene       g ON f.gene_key       = g.gene_key
JOIN   dim_chromosome c ON g.chromosome_key = c.chromosome_key
JOIN   dim_time       t ON f.time_key       = t.time_key
JOIN   dim_disease    d ON f.disease_key    = d.disease_key
GROUP  BY c.chromosome_number, t.year, d.disease_name;

-- Slice: disease = 'Invasive Breast Carcinoma'
SELECT g.gene_name, t.year, d.disease_name, AVG(f.expression_value)
FROM   fact_biological_observation f
JOIN   dim_gene g ON f.gene_key = g.gene_key
JOIN   dim_time t ON f.time_key = t.time_key
JOIN   dim_disease d ON f.disease_key = d.disease_key
WHERE  d.disease_name = 'Invasive Breast Carcinoma'
GROUP  BY g.gene_name, t.year, d.disease_name;
```

Dice composes several `WHERE … IN (…)` clauses; Pivot swaps the `(x, y, z)` axis assignment and triggers a quarter-turn animation on the cube. The SQL that actually ran is exposed through `View SQL` under the cube.

---

# 8. DNA editing pipeline

1. User picks a gene (dropdown of 20 real catalogue genes) and either a known hotspot or a free `(position, alt_base)` pair.
2. Backend pulls the reference window via `GET /alpha/gene-reference`, applies the substitution, retranslates the affected codon with the standard codon table, and classifies the consequence (`silent / conservative missense / missense / nonsense / stop-loss`).
3. Frontend fires two parallel queries:

- `GET /mutations?gene=X` — looks for a warehouse record whose `protein_change` matches the computed `p.XnnY`. On a hit, the row is drawn in accent-red; on a miss, the UI explicitly says *"no warehouse mutation record matches p.XnnY — this is a novel/un-catalogued variant"*.
- `GET /rna-expression?gene=X` — attaches the matching RNA samples; the connected-views banner notes that observed TPM is **independent** of the edit.

4. Variant-effect scorer runs on the user's choice; result labelled as heuristic.
5. Precision Remedy module maps the mutation to FDA-approved therapies (BRAF V600E → Vemurafenib / Dabrafenib; EGFR L858R → Erlotinib / Osimertinib; etc.).

---

# 9. Nexus — natural-language investigation

Full flow (see also `backend/services/nexus.py`):

```
user question
```

```
                ▼
    plan_investigation(question)
      ■■ Groq available ■■  LLM returns JSON plan
      ■■ Groq absent    ■■  deterministic rule-based plan
                ■
                ▼
      _validate_plan(plan)
        drops steps whose tool is not allowlisted,
        drops params that fail typecheck / regex / dim_gene lookup
                ■
                ▼
      execute_plan(session, plan)
        runs each tool against the warehouse (ParamSQL + Pydantic)
                ■
                ▼
    explain_results(question, results)
      ■■ Groq available ■■  grounded prose ≤ 120 words
      ■■ Groq absent    ■■  rule-based summary from structured outputs
```

Allowlisted tools:

```
overview           — warehouse totals
mutations_by_type  — mutation count grouped by type
rna_by_tissue      — mean TPM by tissue × condition (optional gene)
mutations_for_gene — catalogue for one gene
gene_card          — one gene's metadata (coords, biotype, …)
data_sources       — static source provenance
warehouse_health   — connectivity + per-table row counts
```

The LLM can only pick names from this list; it cannot write SQL.

---

# 10. Design system — tokens & typography

## Tokens

```
--canvas:   #F8F5EC   /* page background (warm cream) */
--surface:  #FFFDF8   /* card background */
--sidebar:  #151515   /* floating black sidebar */
--ink:      #191919   /* primary text */
--muted:    #77736C   /* secondary text */
--border:   #E6E0D5   /* subtle warm-grey borders */
--accent:   #7656E8   /* primary action, active selection */
--genomics: #F0B4D5   /* genomics category tint */
--transcript:#F4D96B  /* transcriptomics tint */
--proteomics:#A9C59A  /* proteomics tint */
--imaging:  #AFC8F0   /* vision / ML tint */
```

## Typography

- **Inter** — nav, body, controls, buttons, tables, forms.
- **Fraunces** — selective display face for marketing / masthead.
- **JetBrains Mono** — numbers, SQL, coordinates.

## Patterns

- Rounded cards (14–22 px radius) with 1 px warm border, no heavy shadow.
- Rounded segmented tabs on multi-tab pages (`.seg-tabs`).
- Pastel category stripe on the top edge of module cards.
- Accent purple reserved for primary actions and active selections.

# 11. Security + grounded-answer guarantees

- The LLM never issues raw SQL or Python.
- All tool parameters are validated: string params go through a regex allowlist, gene names are intersected with `dim_gene`.
- All SQL is parameter-bound; no string concatenation on user-controlled values.
- The LLM prompt forbids introducing numbers not in the structured results.
- On any LLM / provider error the system degrades to the deterministic path with an explanation visible to the user.
- No prediction, score, or classification is labelled as clinical evidence; the research disclaimer is on every page footer.

# 12. How to run

## Backend

```
pip install -r backend/requirements.txt
pip install python-multipart
# optional LLM
export GROQ_API_KEY="gsk_…"              # macOS/Linux
$env:GROQ_API_KEY = "gsk_…"              # Windows PowerShell

python -m uvicorn backend.api.main:app --host 127.0.0.1 --port 8000
```

## Frontend

```
cd frontend
npm install
npm run dev        # dev server at http://localhost:3000
npm run build      # production build
```

## ETL + model training (optional — the DB already contains data)

```
python -m warehouse.etl.run_all          # rebuild star schema
python -m ml.classification.train_classifier
python -m ml.clustering.train_clustering
python -m ml.anomaly.train_anomaly
python -m ml.association.train_association
python -m cv.training.train_cnn
```

# 13. Limitations

- **SQLite** is used for ease of reproduction. Scaling to tens of millions of fact rows would need PostgreSQL + parallel ETL.
- The **CNN** has modest capacity on 546 training images; its `76.9 %` test accuracy is a research baseline, not a clinical result.
- The **variant-effect scorer** is a transparent heuristic, not AlphaGenome.

- **DBSCAN** labels every WDBC sample as noise with the current parameters — we expose that honestly rather than tuning it to look pretty.
- **Saved Investigations** is session-scoped (no auth / server persistence) — stated clearly on the page.
- The **data pipeline is a one-time local ingest**, not a live connection to Ensembl / ClinVar / GTEx.

## 14. Reading order for new contributors

1. `warehouse/schema/star_schema.sql` — the single source of truth.
2. `backend/models/orm.py` — SQLAlchemy mirror of the schema.
3. `backend/api/routers/olap_cube.py` — see how an operation becomes SQL.
4. `backend/services/nexus.py` — tool allowlist, planner, explainer.
5. `frontend/components/Sidebar.js` + `AppHeader.js` — the shell.
6. `frontend/app/nexus/page.js` — the signature workspace.
7. `frontend/app/alpha/page.js` + `frontend/components/viz/Dna3DHelix.js` — the DNA bench.
8. `frontend/app/olap/page.js` + `frontend/components/cube/CubeScene.js` — the live cube.

*Document generated on `2026-10-28` from the live backend and database on this instance.*